Vehicular Communications 53 (2025) 100898

Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom

Enhancing task offloading in vehicular networks: A multi-agent cloud-edge-device framework ☆

Peiying Zhang [a,b], Enqi Wang [a], Lizhuang Tan [b,c,*], Neeraj Kumar [d,e], Jian Wang [f], Kai Liu [g,h]

[a] Qingdao Institute of Software, College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China
[b] Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China
[c] Shandong Provincial Key Laboratory of Computing Power Internet and Service Computing, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China
[d] Department of Computer Science and Engineering, Thapar University, Patiala 147004, India
[e] School of Computer Science & Technologies, University of Economics and Human Sciences, Warsaw 01-043, Poland
[f] College of Science, China University of Petroleum (East China), Qingdao 266580, China
[g] State Key Laboratory of Space Network and Communications, Tsinghua University, Beijing 100084, China
[h] Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China

ARTICLE INFO

Keywords:
Internet of vehicles
Mobile edge computing
Deep reinforcement learning
Task offloading
Resource allocation

ABSTRACT

In vehicular networks, the increasing demand for computational resources often exceeds the capabilities of in-vehicle devices. To address these challenges, we propose a cloud-edge-device collaborative framework integrated with a Multi-Agent Deep Reinforcement Learning (MADRL) algorithm for dynamic optimization of task offloading and resource allocation. Experimental evaluations demonstrate the proposed algorithm's superiority over traditional methods, achieving an 11% reduction in energy consumption and a 23% increase in task completion rate compared to local processing-only strategies, while reducing average task delay by 50% relative to static offloading approaches. The MADRL-based framework not only ensures efficient task distribution but also adapts to fluctuating network conditions, achieving a resource utilization rate of 85%. These findings underscore its potential to enhance performance in intelligent transportation systems by balancing computational efficiency, energy consumption, and task latency.

1. Introduction

With the rapid development of intelligent connected technologies, vehicular networks have become a cornerstone of smart transportation and autonomous driving. These networks integrate vehicles, infrastructure, and cloud systems to enable real-time data transmission and analysis, enhancing transportation system efficiency and intelligence. However, as applications become increasingly complex, the computational demands of tasks such as image recognition, data analysis, and path planning impose significant pressure on onboard devices. Meeting these demands requires high real-time performance and low latency, which many vehicle terminals cannot achieve with their limited resources.

Traditional vehicular network architectures rely primarily on local computational capabilities; however, this approach has inherent limitations. As task complexity increases, onboard devices struggle to process computation-intensive tasks, leading to resource bottlenecks. Prolonged high-load operations also result in excessive energy consumption, reducing vehicle range and hardware lifespan. These challenges underline the

☆ This work is partially supported by the National Natural Science Foundation of China under Grant 62471493 and 62402257, partially supported by the Natural Science Foundation of Shandong Province under Grant ZR2023LZH017, ZR2024MF066, and 2023QF025, partially supported by the Open Foundation of Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences) under Grant 2023ZD010.

* Corresponding author at: Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China.
E-mail addresses: zhangpeiying@upc.edu.cn (P. Zhang), z23070086@s.upc.edu.cn (E. Wang), tanlzh@sdas.org (L. Tan), neeraj.kumar@thapar.edu (N. Kumar), wangjiannl@upc.edu.cn (J. Wang), liukaiv@tsinghua.edu.cn (K. Liu).

https://doi.org/10.1016/j.vehcom.2025.100898
Received 12 January 2025; Received in revised form 9 February 2025; Accepted 21 February 2025

Available online 25 February 2025

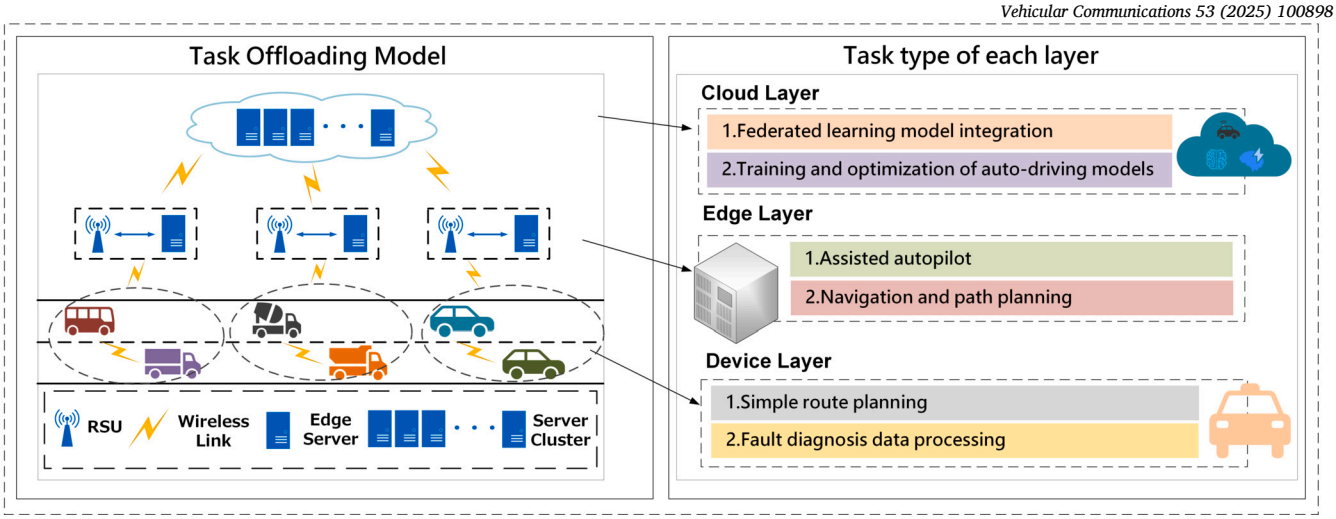


Fig. 1. Task Offloading Model in Cloud-Edge-Device Collaborative Architecture for IoV.

urgent need for efficient strategies to distribute the computational load and enhance the performance of vehicular networks.

To address these limitations, task offloading has emerged as a promising solution. By transferring computational tasks to more capable computing nodes, such as edge or cloud servers, offloading reduces the load on vehicle terminals and improves execution efficiency. However, existing offloading strategies often focus on single-tier architectures, either cloud or edge, which limits their ability to address diverse task requirements. Cloud servers provide abundant computational resources but suffer from high transmission delays, while edge servers offer lower latency but have limited capacity. In real-world scenarios, different tasks demand varying levels of computational resources and latency, making a single-tier approach inadequate.

Previous research has explored various task offloading strategies to mitigate these issues. Heuristic methods [18,29] simplify decision-making for offloading, while game-theoretic models [17,24] optimize offloading under resource constraints. Multi-criteria optimization techniques [9,23] balance objectives such as energy efficiency and latency. Despite advancements, these methods often focus on static or narrowly defined scenarios and fail to adapt to the dynamic and heterogeneous nature of vehicular networks. The NP-hard nature of the task offloading problem exacerbates these challenges, leading to scalability issues and suboptimal performance in large-scale, dynamic environments.

Dynamic connectivity in vehicular networks further complicates task offloading. Vehicle mobility results in constantly changing network conditions, undermining the effectiveness of fixed-strategy schemes. These schemes often fail to adapt to fluctuating demands and underutilize idle resources among vehicles, resulting in inefficiencies and poor coordination. This lack of flexibility limits their ability to meet the real-time demands of modern vehicular applications.

To address these challenges, we assume a cloud-edge-device collaborative architecture designed to meet the dynamic and resource-intensive demands of vehicular networks. As illustrated in Fig. 1, the architecture is divided into three layers based on task types: the central cloud layer, the edge layer, and the device layer.

In the central cloud layer, high-computation tasks are processed, such as model integration and autonomous driving model optimization. The edge layer manages latency-sensitive tasks, such as assisting driving systems and navigation/path planning. The terminal layer, which is closest to the vehicles, handles less complex tasks, such as simple route planning and real-time vehicle engine diagnostics.

Vehicles in the device layer generate various tasks during their operation. Due to the limited energy and computational resources of the vehicles, some computational tasks need to be offloaded to the edge or central cloud servers via Vehicle-to-Infrastructure (V2I) communication for assistance in computation. The mobile edge computing (MEC) servers in the edge layer obtain real-time vehicle status information through roadside units. Their faster request-response speeds can provide quicker task processing for in-vehicle devices compared to the cloud center.

The central cloud layer consists of multi-core cloud servers. Although the distance from the device layer can cause some data transmission delays, its ample computational power can handle tasks with high computational requirements and long processing times in vehicles. Additionally, in certain special circumstances, nearby vehicles can also serve as targets for task offloading to balance the overall utilization of system resources.

By leveraging the complementary advantages of these layers, the architecture ensures efficient and reliable task offloading.

At the core of this architecture is a MADRL algorithm that dynamically optimizes offloading decisions based on real-time network states, task priorities, and resource availability. This approach enhances system efficiency, scalability, and adaptability in complex vehicular environments.

The key contributions of this research are summarized as follows:

1. **Dynamic Optimization**: A MADRL-based framework adjusts offloading strategies in real time, considering network states, task requirements, and resource constraints to significantly enhance efficiency.
2. **Hierarchical Resource Utilization**: The proposed architecture leverages the combined strengths of cloud, edge, and end layers to allocate resources effectively and improve overall performance.
3. **Adaptability and Scalability**: Designed for dynamic vehicular environments, the framework accommodates fluctuating mobility and network conditions while scaling effectively as vehicular density increases.

The remainder of this paper is organized as follows: Section 2 reviews related work on task offloading in vehicular networks. Section 3 presents the task offloading model and key challenges. Section 4 details the MADRL-based approach. Section 5 provides simulation results, and Section 6 concludes the study.

2. Related work

2.1. IoV computation offloading architectures

Computation offloading architectures in the IoV have evolved to address diverse demands. [10] proposed a cloud-centric approach, lever-

aging the cloud's powerful computational resources to handle offloading tasks. To mitigate resource scarcity in MEC servers caused by high offloading demands, [1] introduced a collaborative architecture integrating cloud centers, edge servers, and backup servers, significantly enhancing computational capacity.

Zheng et al. [30] explored the integration of vehicular and cloud resources, improving offloading efficiency by utilizing vehicular resources alongside cloud computing. Similarly, [28] proposed a three-tier framework comprising cloud servers, MEC servers, and mobile users, which reduces latency and enhances resource utilization by offloading tasks to edge servers.

2.2. Performance optimization in computation offloading

Performance optimization in computation offloading for IoV involves key metrics such as energy consumption, latency, and security. [11] proposed a multi-timescale MEC framework to minimize energy consumption by jointly allocating caching and computational resources at both user and server sides. This framework dynamically adapts to IoV mobility by adjusting resource allocation strategies based on the changing network environment.

[21] introduced an artificial fish swarm algorithm to optimize energy consumption during task execution and communication. By simulating social behavior, the algorithm identifies optimal offloading strategies that balance energy efficiency and task execution performance. Similarly, [10] proposed a joint offloading scheme leveraging both vehicular and central clouds, reducing latency, resource consumption, and energy use by combining the low-latency benefits of vehicular clouds with the computational power of central clouds.

For latency-sensitive tasks, [15] designed an SDN-based MEC architecture that uses software-defined networking to optimize traffic control, ensuring fast response times. For latency-insensitive services, [26] proposed utilizing idle bandwidth from IoV or IoT to offload non-sensitive tasks, improving resource utilization and reducing processing delays.

To address security concerns, [6] explored NOMA-based and multi-access assisted offloading schemes, enhancing data transmission security and privacy, thereby improving overall offloading reliability in IoV environments.

2.3. Optimization algorithms for computation offloading

Optimization algorithms for computation offloading in IoV have leveraged both traditional and modern techniques. [22] employed game theory to model competition among vehicles for MEC server resources, introducing a distributed incentive mechanism that achieves a Nash equilibrium, ensuring optimal decisions for all participants.

Graph theory has been widely used to solve shortest-path problems and optimize task offloading. [20] applied graph theory to cloud-assisted offloading, optimizing traversal algorithms for V2I and Vehicle-to-Vehicle (V2V) links to maximize system throughput in IoV.

Heuristic algorithms are valued for their efficiency in complex optimization scenarios. [7] used the artificial fish swarm algorithm to jointly optimize computation and communication resources, reducing energy consumption in 5G networks. Similarly, [19] employed an improved genetic algorithm to minimize delay in offloading tasks, iteratively refining solutions through selection, crossover, and mutation. To address dynamic IoV environments, [13] applied the multi-armed bandit algorithm, effectively handling challenges posed by vehicle mobility, fluctuating wireless channels, and varying computing resources.

Other studies, such as [14] and [4], optimized offloading strategies by considering user attributes like geographic location and physical neighbors. While these methods provide valuable insights, they are often limited to simpler IoV scenarios. As IoV environments grow increasingly complex, further innovation in optimization algorithms is needed to address emerging challenges.

2.4. Deep reinforcement learning for computation offloading

Deep reinforcement learning (DRL) has gained attention for solving decision-making problems in dynamic environments. By optimizing strategies based on environmental feedback, DRL enables agents to achieve long-term optimal rewards without prior knowledge. In the dynamic IoV environment, where traditional methods often offer approximate solutions with high overhead, DRL presents a promising alternative for edge computing task offloading.

For instance, [25] proposed a deep Q-learning-based task offloading scheme that adapts to rapid IoV changes, demonstrating enhanced adaptability and performance. Similarly, [3] optimized energy consumption and load balancing in roadside units using an online deep Q-learning technique. [12] improved service quality by allocating resources between vehicles and servers through Q-learning. To reduce energy consumption and task delays, [8] integrated deep learning with reinforcement learning, enhancing traditional Q-learning algorithms. Additionally, [27] developed a DRL-based strategy that considers task dependency and vehicle mobility, utilizing simulated annealing to minimize offloading failures and energy costs. Other works, such as [5] and [16], focused on optimizing latency, computational costs, and resource allocation through advanced Q-learning techniques.

Despite the success of Q-learning and DQN in DRL, these methods struggle with large-scale problems, such as storing vast state-action pairs and handling continuous action spaces. Given the complexity of IoV environments, advanced DRL algorithms are required for long-term optimization. This paper addresses task offloading and resource optimization using a cloud-edge-device collaboration framework. By integrating DRL into a multi-agent model, we aim to improve offloading efficiency and adaptability in IoV environments.

3. Task offloading model of IoV

This section discusses the modeling process, covering the communication, computational, and system cost models.

Fig. 1 illustrates the cloud-edge-device collaborative architecture in vehicular networks, consisting of three layers: the vehicle terminal layer, the edge cloud cluster layer, and the central cloud cluster layer. The vehicle terminal layer generates tasks that can be executed locally or offloaded to either the edge cloud or central cloud. The central cloud is a large-scale server cluster with significant computational resources. The edge cloud cluster layer includes E edge clusters, denoted as $EC = \{ec_1, ec_2, \ldots, ec_E\}$, where each cluster serves V vehicle terminals. Each edge cloud cluster ec_e is equipped with a base station that can forward data to the edge server, central cloud, or local terminals.

Each edge cloud cluster ec_e consists of U virtual machines (VMs), denoted as $VM^e = \{vm_1^e, vm_2^e, \ldots, vm_U^e\}$, with each VM represented by a five-tuple: $vm_u^e = \{cpu_{e,u}^{total}, cpu_{e,u}^{rest}, ram_{e,u}^{total}, ram_{e,u}^{rest}, \varepsilon_{e,u}\}$. The attributes are as follows: $cpu_{e,u}^{total}$ is the total CPU, $cpu_{e,u}^{rest}$ is the remaining CPU, $ram_{e,u}^{total}$ is the total RAM, $ram_{e,u}^{rest}$ is the remaining RAM, and $\varepsilon_{e,u}$ is the cost coefficient per unit time. Vehicle terminals ter_e^v have a four-tuple $\{cpu_{e,v}^{total}, cpu_{e,v}^{rest}, ram_{e,v}^{total}, ram_{e,v}^{rest}\}$ with similar resource attributes.

3.1. Task type

In vehicular networks, tasks vary widely in their requirements. We assume a time-slotted offloading decision structure, with each time frame denoted as $t \in \{0, 1, 2, \ldots, T\}$. Let $Asign_{v,t} = \{\phi_v^s, \phi_v^c, \phi_v^d\}$ represent a task generated by vehicle v at time t, where ϕ_v^s is the task size, ϕ_v^c the required computational resources, and ϕ_v^d the maximum tolerable delay. Tasks are categorized into three types:

1. **High-Priority Tasks**: Delay-sensitive tasks, such as collision warnings or emergency braking, requiring $\phi_v^d < Thd_1$. These are best executed locally on the vehicle.

2. **Medium-Priority Tasks**: Tasks like adaptive cruise control with moderate delay tolerance, $Thd_1 \leq \phi_v^d < Thd_2$, suitable for offloading to MEC servers or nearby vehicles.
3. **Low-Priority Tasks**: Less delay-sensitive tasks, such as multimedia streaming, with $\phi_v^d \geq Thd_2$ and higher computational needs, ideal for cloud execution.

This classification, combined with a cloud-edge-device architecture, optimizes system offloading and processing efficiency.

3.2. Communication model

Vehicular communication systems utilize two primary modes: V2V and V2I. V2V typically employs Dedicated Short-Range Communication (DSRC), while V2I leverages LTE-Advanced (LTE-A) networks. Both modes are influenced by multipath fading due to vehicle mobility, which is effectively modeled using the Rayleigh fading model.

In real - world vehicular network scenarios, shadow fading and noise interference significantly impact communication performance. Shadow fading, resulting from signal obstruction by obstacles such as buildings or terrain features, causes additional random signal attenuation. It follows a log - normal distribution, denoted as $S \sim \mathcal{N}(0, \sigma_S^2)$, with $\sigma_S = 8\ dB$ in this study. Noise interference, originating from various sources like other wireless transmitters and environmental noise, also plays a crucial role in degrading signal quality.

The path loss for V2V and V2I communications at time t is defined as:

$$loss_v^{V2V} = 10^{\frac{-63.3+17.7\lg\left(d_{v,TER}(t)\right)+S}{10}}, \tag{1}$$

$$loss_v^{V2I} = 10^{\frac{-63.3+17.7\lg\left(d_{v,E}(t)\right)+S}{10}}. \tag{2}$$

Here, S is the shadow fading factor. $d_{v,TER}(t)$ and $d_{v,E}(t)$ represent the distances from vehicle v to an offloadable vehicle (OV) and an edge cloud cluster (ECC), respectively, and R_{max} is the maximum V2V communication range. These distances can be expressed as:

$$d_{v,OV}(t) = \|p_v(t) - p_{OV}(t)\|, \tag{3}$$

$$d_{v,ECC}(t) = \|p_v(t) - p_{ECC}(t)\|, \tag{4}$$

where $p_v(t)$, $p_{OV}(t)$, and $p_{ECC}(t)$ are the respective positions of the vehicle, OV, and ECC.

The uplink transmission rates for V2V and V2I communication also need to account for both shadow fading and noise interference. The uplink transmission rates for V2V and V2I communication are:

$$Rate_{v,OV}^{V2V} = BW_{V2V} \log_2\left(1 + \frac{PW_{v,loss}^{V2V}\|h^2\|}{I + N_0}\right), \tag{5}$$

$$Rate_{v,ECC}^{V2I} = BW_{V2I} \log_2\left(1 + \frac{PW_{v,loss}^{V2I}\|h^2\|}{I + N_0}\right), \tag{6}$$

where BW_{V2V} and BW_{V2I} denote the respective bandwidths, PW_v represents the vehicle's transmission power, N_0 is the Gaussian noise power, I is the interference noise power and h is the Rayleigh fading coefficient.

The path loss for both communication modes can alternatively be represented in dB form:

$$PL(d) = PL_0 + 10\sigma \log_{10}(d) + S, \tag{7}$$

where PL_0 is the path loss at a reference distance, and σ is the path loss exponent.

The downlink transmission rates for V2V and V2I are expressed as:

$$Rate_{OV,v}^{V2V} = BW_{V2V} \log_2\left(1 + \frac{P_{OV,loss}^{V2V}\|h^2\|}{I + N_0}\right), \tag{8}$$

$$Rate_{ECC,v}^{V2I} = BW_{V2I} \log_2\left(1 + \frac{P_{ECC,loss}^{V2I}\|h^2\|}{I + N_0}\right), \tag{9}$$

where P_{OV} and P_{ECC} are the transmission powers of the OV and ECC, respectively.

The average uplink and downlink rates for V2V and V2I communication are computed as:

$$\overline{Rate_{v,OV}^{V2V}} = \frac{\int_0^{t_{v,OV}^{V2V}} Rate_{v,OV}^{V2V}(t)\,dt}{t_{v,OV}^{V2V}}, \tag{10}$$

$$\overline{Rate_{v,ECC}^{V2I}} = \frac{\int_0^{t_{v,ECC}^{V2I}} Rate_{v,ECC}^{V2I}(t)\,dt}{t_{v,ECC}^{V2I}}, \tag{11}$$

$$\overline{Rate_{V2V}^{v,OV}} = \frac{\int_0^{t_{v,OV}^{V2V}} Rate_{V2V}^{v,OV}(t)\,dt}{t_{v,OV}^{V2V}}, \tag{12}$$

$$\overline{Rate_{V2I}^{v,ECC}} = \frac{\int_0^{t_{v,ECC}^{V2I}} Rate_{V2I}^{v,ECC}(t)\,dt}{t_{v,ECC}^{V2I}}. \tag{13}$$

This model comprehensively describes the dynamic nature of vehicular communication, incorporating multipath fading, path loss, and transmission rates, which are critical for ensuring reliable task offloading and resource allocation.

3.3. Computation model

In vehicular networks, vehicles often prioritize task offloading to external entities to reduce their computational burden, potentially causing imbalances in resource utilization. To address this, we propose an optimization strategy within a cloud-edge-device collaborative framework to minimize system latency, energy consumption, and inefficiencies. Task offloading strategies are categorized into four types, with latency and energy consumption defined as follows:

Local processing When a task is executed locally, the vehicle allocates its computational resources $f_{t,v}$ without incurring transmission latency or energy consumption. The energy consumption is proportional to the square of the CPU voltage:

$$E_v^{LOC} = \eta_v (f_{t,v})^2 \phi_v^c, \tag{14}$$

where η_v depends on the vehicle's CPU architecture. The computational delay is expressed as:

$$T_v^{LOC} = \frac{d_v^u \phi_v^s}{f_{t,v}}, \tag{15}$$

where d_v^u denotes CPU cycles required per data unit, and ϕ_v^s is the task size.

Offloading to ECC When offloading to an ECC, the total energy consumption includes both transmission and computation:

$$\begin{aligned} E_{v,e}^{ECC} &= E_{v,e}^{Trans} + E_e^{LOC} \\ &= \frac{PW_{v,loss}^{V2I}\phi_v^d}{Rate_{v,ECC}^{V2I}} + \phi_n^c C_e, \end{aligned} \tag{16}$$

where $E_{v,e}^{Trans}$ is the transmission energy, and C_e represents the ECC's energy consumption per computational unit. The total delay includes computational and transmission components:

$$T_{v,e}^{ECC} = T_e^{LOC} + T_{v,e}^{Trans}, \tag{17}$$

where

$$T_e^{LOC} = \frac{d_e^u \phi_v^s}{f_{t,e}}, \quad T_{v,e}^{Trans} = \frac{\phi_v^d}{Rate_{v,ECC}^{V2I}}. \tag{18}$$

Offloading to central cloud Offloading low-priority tasks to the central cloud benefits from its robust computational capacity, rendering computational energy consumption negligible. However, energy and delay from data transmission remain significant:

$$E_{v,c}^{CCS} = \frac{PW_{v,loss}^{V2I} \phi_v^d}{Rate_{v,ECC}^{V2I}} + \frac{PW_{ECC,loss}^{V2I} \mathcal{R}_v}{Rate_{ECC,v}^{V2I}}, \tag{19}$$

$$T_{v,c}^{CCS} = T_{v,c}^{uplink} + T_{c,v}^{downlink}, \tag{20}$$

where $T_{v,c}^{uplink} = \frac{\phi_v^d}{Rate_{v,ECC}^{V2I}}$ and $T_{c,v}^{downlink} = \frac{\mathcal{R}_v}{Rate_{ECC,v}^{V2I}}$.

Offloading to nearby OV Offloading tasks to a nearby OV shares similarities with offloading to ECCs. The total energy consumption is:

$$\begin{aligned} E_{v,o}^{OV} &= E_{v,o}^{Trans} + E_o^{LOC} \\ &= \frac{PW_{v,loss}^{V2V} \phi_v^d}{Rate_{v,OV}^{V2V}} + \eta_o (f_{t,o})^2 \phi_o^c, \end{aligned} \tag{21}$$

and the total delay is:

$$\begin{aligned} T_{v,o}^{OV} &= T_{v,o}^{Trans} + T_o^{LOC} \\ &= \frac{\phi_v^d}{Rate_{v,OV}^{V2V}} + \frac{d_o^u \phi_v^s}{f_{t,o}}. \end{aligned} \tag{22}$$

This model comprehensively accounts for latency and energy trade-offs in different offloading strategies, providing a basis for optimizing task execution in vehicular networks.

3.4. System cost model

Building on the task priority classification established in Section 3.1, we formulate four distinct offloading strategies to meet the heterogeneous requirements of vehicular tasks. Task offloading strategies are classified into four categories: 1) local processing, denoted as δ_l; 2) offloading to the ECC, denoted as δ_e; 3) offloading to cloud servers, denoted as δ_c; and 4) offloading to nearby OV, denoted as δ_o. Each category incurs different system costs depending on task priority and resource requirements.

For high-priority tasks, system cost is dominated by local computation time and energy consumption. Medium-priority tasks offloaded to ECCs or nearby OVs involve additional costs from task transfer, including transmission delays and energy expenditure. For low-priority tasks offloaded to cloud servers, the cost further includes processing result transmission due to larger data volumes. Thus, system cost integrates both energy consumption and time delay components.

The execution time for a task generated by vehicle v at time t is expressed as:

$$\begin{aligned} Time_{v,t} = {} & \delta_l T_v^{LOC} + \delta_e T_{v,e}^{ECC} + \delta_o T_{v,o}^{OV} \\ & + \delta_c T_{v,c}^{CCS} + T_{q,x}^v, \end{aligned} \tag{23}$$

where $\delta_l, \delta_e, \delta_o, \delta_c \in \{0, 1\}$ indicate the selected offloading method, and $T_{q,x}^v$ represents the task's waiting time in the queue.

Energy consumption for task execution is:

$$\begin{aligned} Energy_{v,t} = {} & \delta_l E_v^{LOC} + \delta_e E_{v,e}^{ECC} \\ & + \delta_o E_{v,o}^{OV} + \delta_c E_{v,c}^{CCS}. \end{aligned} \tag{24}$$

The total system cost combines time and energy as:

$$Cost_{v,t} = \varepsilon_v Time_{v,t} + (1 - \varepsilon_v) Energy_{v,t}, \tag{25}$$

where $\varepsilon_v \in [0, 1]$ balances the relative importance of time delay and energy consumption in the overall cost.

This model provides a comprehensive framework to evaluate the cost of various offloading strategies, facilitating efficient resource utilization and task prioritization in vehicular networks.

3.5. Problem formulation

This paper introduces a cloud-edge-device collaborative architecture to address load balancing in IoV scenarios. The proposed framework requires efficient task offloading and resource allocation strategies to optimize computational resource utilization, reduce energy consumption and latency, and enhance system stability.

To improve user experience at vehicle terminals, performance metrics include task completion count, load rate, and system cost (comprising latency and energy consumption). These metrics are integrated into the following objective function:

$$P_t = \sum_{v \in V} \frac{\mathcal{T}_{v,t} \cdot (1 - \theta L_{v,t})}{Cost_{v,t} \cdot (1 + \phi L_{v,t})}, \tag{26}$$

where $\mathcal{T}_{v,t}$ denotes the number of tasks completed by vehicle v at time t, $Cost_{v,t}$ represents the system cost, $L_{v,t}$ is the vehicle's load rate, and θ, ϕ are weighting factors adjusting the impact of load rate on system cost and task completion.

The optimization problem aims to maximize P_t, subject to the following constraints:

$$\begin{aligned} &\max \quad P_t \\ &s.t. \\ &(C1) \quad \delta_l, \delta_e, \delta_o, \delta_c \in \{0, 1\} \\ &(C2) \quad (\delta_l + \delta_e + \delta_o + \delta_c) \leq 1 \\ &(C3) \quad \eta \in [0, 1] \\ &(C4) \quad \sum_{v \in \mathcal{V}} x_{v,c}^{V2I,\text{uplink}} \leq 1 \quad \forall c \in C_{V2I} \\ &(C5) \quad \sum_{v \in \mathcal{V}} x_{v,c}^{V2I,\text{downlink}} \leq 1 \quad \forall c \in C_{V2I,\text{downlink}} \\ &(C6) \quad \sum_{k \in \mathcal{T}} y_{v,t} \leq 1 \quad \forall v \in \mathcal{V} \\ &(C7) \quad Time_{v,t} \leq \phi_n^d \end{aligned} \tag{27}$$

Here, constraint $(C1)$ defines the four task execution modes, and $(C2)$ ensures only one mode is selected per task. Constraint $(C3)$ limits resource allocation to available capacities. Constraints $(C4)$ and $(C5)$ handle uplink and downlink channel allocation for V2I communication, where $x_{v,c}^{V2I,\text{uplink}}$ and $x_{v,c}^{V2I,\text{downlink}}$ are binary indicators of channel assignment. Constraint $(C6)$ ensures tasks are assigned to a single vehicle, and $(C7)$ enforces that execution time does not exceed the task's delay tolerance.

By implementing these constraints, the proposed framework enables seamless collaboration among cloud, edge, and terminal devices, optimizing resource allocation and task execution to enhance system performance and user experience in IoV environments.

4. Proposed algorithm

In the IoV environment, in-vehicle devices' strategies evolve dynamically during training, which reduces system stability. Additionally, constrained resources lead to intense competition among devices, making traditional single-agent reinforcement learning algorithms inadequate for the complex scenarios in VANETs. Balancing system cost minimization, task completion maximization, and equitable load distribution presents significant challenges in resource allocation and task offloading. To address these, we propose a MADRL algorithm.

By framing the optimization problem, task offloading and resource allocation are modeled as a Markov Decision Process (MDP). The MADRL algorithm is used to train the network architecture within a cloud-edge-device collaborative framework, optimizing task offloading and resource allocation in a multi-agent environment. During training, each agent interacts with the environment, considering the actions of others, and updates its model to derive optimal strategies for task offloading and resource allocation. The following sections define the MDP elements, including state space, action space, and reward function.

4.1. State

At the beginning of each time slot, agents on devices at various layers (including vehicle terminals, edge servers, and cloud servers) collect the states and available resources of all vehicles within their communication range, excluding themselves, to establish a state information table. At time t, the collected state information table consists of the following elements:

1. **Task State**: The task states of vehicles within the communication range are denoted as $S_t^{TaskS} = (Asign_{1,t}, Asign_{2,t}, \dots, Asign_{v,t}, \dots, Asign_{V,t})$. If a vehicle is outside the communication range, its task state is set to $Asign_v = 0$.
2. **Computing Resource State**: At time t, the available computing resources of the OV are denoted as $S_{o,t}^{CompR} = f_{t,o}$, those of the ECC are denoted as $S_{e,t}^{CompR} = f_{t,e}$, and those of the cloud server are denoted as $S_{c,t}^{CompR} = f_{t,c}$.
3. **Communication Resource State**: At time t, the available communication resources of the OV are denoted as $S_{o,t}^{CommR}$, those of the ECC are denoted as $S_{e,t}^{CommR}$, and those of the cloud server are denoted as $S_{c,t}^{CommR}$.

Based on the above definitions, the state of the OV at time t can be expressed as

$$S_t^o = (S_t^{TaskS}, S_{o,t}^{CompR}, S_{o,t}^{CommR}), \tag{28}$$

the state of the ECC at time t can be expressed as

$$S_t^e = (S_t^{TaskS}, S_{e,t}^{CompR}, S_{e,t}^{CommR}), \tag{29}$$

and the state of the cloud server at time t can be expressed as

$$S_t^c = (S_t^{TaskS}, S_{c,t}^{CompR}, S_{c,t}^{CommR}). \tag{30}$$

Therefore, the overall system state space at time t can be defined as

$$S_t = (S_t^{1,o}, \dots, S_t^{O,o}, S_t^{1,e}, \dots, S_t^{E,e}, S_t^{1,c}, \dots, S_t^{C,c}). \tag{31}$$

4.2. Action

When there are pending tasks in the task queue that need to be offloaded, tasks can be categorized into four offloading directions based on their types, as described earlier. For these four decision directions, the agent needs to determine the task execution method at time t and allocate the corresponding computing and communication resources to the vehicle. As previously mentioned, the binary variables $\delta_l, \delta_e, \delta_o, \delta_c \in \{0, 1\}$ represent the four task offloading directions. Since a task can only be offloaded in one direction, the constraints defined in Equation (27) (C2) must be satisfied, i.e., $(\delta_l + \delta_e + \delta_o + \delta_c) \le 1$.

When a task needs to be offloaded, only one of the binary variables $\delta_l, \delta_e, \delta_o, \delta_c$ will take a value of 1, depending on the selected offloading direction. For local processing, offloading to OV, or offloading to ECC, the allocated computing resources can be denoted as ϖ, and the allocated communication resources can be denoted as $x_{v,x}^{V2I,\text{uplink}}$. However, as analyzed in Section 3.3, when a task is offloaded to the cloud server, the vast computing resources of the cloud server make it capable of handling low-priority tasks, allowing us to ignore the computing resources allocated during such offloading. Nevertheless, since the computation results returned from the cloud server typically require more communication resources, the allocated communication resources in this case can be denoted as $x_{v,c}^{V2I,\text{uplink}}, x_{v,c}^{V2I,\text{downlink}}$.

In summary, the action space of the agent at time t can be represented as

$$A_t^{agent} = (\delta_l, \delta_e, \delta_o, \delta_c, \varpi, x_{v,x}^{V2I,\text{uplink}}, x_{v,c}^{V2I,\text{uplink}}, x_{v,c}^{V2I,\text{downlink}}), \tag{32}$$

and the overall system action space can be defined as

$$A_t = (A_{1,t}^{agent}, A_{2,t}^{agent}, \dots, A_{X,t}^{agent}). \tag{33}$$

4.3. Reward function

The reward function is a cornerstone of the proposed MADRL framework, aligning task offloading and resource allocation with system constraints while driving performance optimization. It integrates multiple objectives, including energy efficiency, task completion, delay reduction, and load balancing, while penalizing resource overload to ensure system stability.

The reward function is composed of the following components, each targeting a specific aspect of system performance:

Energy efficiency reward Minimizing energy consumption is a primary goal of the proposed framework. The total energy consumption, denoted as $\text{Energy}_{v,t}$, is calculated using Equation (24) along with constraints (C1) and (C2). The energy efficiency reward is defined as:

$$R_{\text{energy}} = \mu_1 \cdot \frac{\sum_{n \in \mathcal{N}} \text{Energy}_{v,t}^n}{|\mathcal{N}|}, \tag{34}$$

where $|\mathcal{N}|$ is the total number of tasks completed during the current time step, and μ_1 is a weighting factor penalizing energy consumption.

Task completion reward To encourage task completion and balance energy efficiency with the number of completed tasks, the task completion reward is expressed as:

$$R_{\text{completion}} = \mu_2 \cdot \left| |\mathcal{N}| - N_{\min} \right|, \tag{35}$$

where μ_2 is the weight assigned to task completion, $|\mathcal{N}|$ is the number of tasks completed within the current time step, and $N_{\min}$ represents the minimum required number of completed tasks.

Latency penalty Tasks exceeding the allowable delay threshold ϕ_v^d incur a penalty to ensure timely task execution. The latency penalty is defined as:

$$R_{\text{latency}} = \mu_3 \cdot \sum_{n \in \mathcal{N}} \max(0, \text{Time}_{v,t}^n - \phi_v^d), \tag{36}$$

where $\text{Time}_{v,t}^n$ denotes the actual delay of task n, ϕ_v^d is the delay threshold, and μ_3 is the penalty weight. It can be seen from the above formula that the longer the actual task delay $\text{Time}_{v,t}^n$ is, that is, the more the delay exceeds the maximum acceptable delay ϕ_v^d, the higher the penalty will be.

Overload penalty To prevent system overload, ensure stable performance, and avoid intense resource competition among agents, an adaptive overload penalty mechanism is introduced. The overload penalty coefficient R_{overload} is defined as:

$$R_{\text{overload}} = \mu_4 \cdot \sum_{i=1}^{n} O_v \cdot \begin{cases} e^{L_{\text{avg}} - 0.9}, & \text{if } L_{\text{avg}} > 0.9, \\ 1, & \text{if } L_{\text{avg}} \le 0.9, \end{cases} \tag{37}$$

Algorithm 1 MADRL Training Process.

Initialize: Initialized actor networks μ_{θ_i}, critic networks Q_{μ_i}, target networks μ'_{θ_i}, Q'_{μ_i}, replay buffer D, batch size X, learning rate α, discount factor γ, soft update coefficient τ.

while not converged **do**
  for each agent i **do**
    Observe state S_i, select action $a_i = \mu_{\theta_i}(S_i) + \mathcal{N}$.
    Execute a_i, receive reward r_i, observe S'_i.
    Store transition (S, a, r, S') in D.
  end for
  if enough samples in D **then**
    Sample mini-batch (S_j, a_j, r_j, S'_j) from D.
    for each agent i **do**
      Compute target actions $a'_i = \mu'_{\theta_i}(S'_i)$ and target Q-value:

$$y_j = r_{i,j} + \gamma Q'_{\mu_i}(S'_j, a'_1, \ldots, a'_I).$$

      Update critic by minimizing:

$$L(\theta_i) = \frac{1}{X}\sum_{j=1}^{X}\left(y_j - Q_{\mu_i}(S_j, a_j)\right)^2.$$

      Update actor network: $\theta_i \leftarrow \theta_i + \alpha \nabla_{\theta_i} J(\mu_{\theta_i})$.
    end for
    Update target networks:

$$\mu'_{\theta_i} \leftarrow \tau\mu_{\theta_i} + (1-\tau)\mu'_{\theta_i}, \quad Q'_{\mu_i} \leftarrow \tau Q_{\mu_i} + (1-\tau)Q'_{\mu_i}.$$

  end if
end while
return Actor networks μ_{θ_i} for all agents.

where μ_4 is the base penalty coefficient, O_v represents the overload amount for vehicle v, and L_{avg} is the average system load rate. The overload amount O_v is computed as:

$$O_v = \max(0, \text{Task}_v - \text{Buffer}_v), \tag{38}$$

where Task_v is the number of tasks assigned to vehicle v, and Buffer_v is the buffer capacity of vehicle v. The average system load rate L_{avg} is calculated as:

$$L_{\text{avg}} = \frac{1}{V}\sum_{v=1}^{V}\frac{C^v_{\text{current}}}{C^v_{\max}}, \tag{39}$$

where C^v_{current} and $C^v_{\max}$ denote the current computational load and maximum computational capacity of vehicle v, respectively, and V is the total number of vehicles.

When the average load rate exceeds 90%, an exponential term significantly increases the penalty to discourage system overload. This adaptive design helps maintain system stability and ensures efficient resource utilization. Under normal load conditions ($L_{\text{avg}} \leq 0.9$), a basic linear penalty is applied to mitigate overload occurrences.

Total reward function The overall reward function is a weighted combination of the aforementioned components:

$$R = -1 \cdot \left(R_{\text{energy}} + R_{\text{completion}} + R_{\text{latency}} + R_{\text{overload}}\right). \tag{40}$$

4.4. Joint resource optimization method

This paper presents a MADRL framework designed to address the challenges of task offloading and resource allocation in cloud-edge-device collaborative vehicular networks. The proposed framework consists of two primary components: the actor network and the critic network. The actor network is responsible for determining the optimal action a_i for each agent i based on its observed state S_i, while the critic network evaluates the action's quality through a centralized state-action value function $Q_{\mu_i}(S, a_1, \ldots, a_I)$.

In this framework, an experience replay buffer is employed to store a certain amount of data related to the agent's previous state transitions in the environment. When the network model requires updates, data is randomly sampled from the experience replay buffer for training. This approach breaks the correlation between data samples and stabilizes the training process.

The training process of MADRL includes two main components: centralized training and decentralized execution. During centralized training, training samples are randomly selected from the experience replay buffer D in the cloud center. Each participating agent i observes its local environment and inputs the state information into the actor network to obtain the optimal action a_i.

The critic network evaluates the action values based on the states S and actions a of all agents in the environment. Subsequently, the parameters of both networks are updated according to the loss function. In the decentralized execution phase, agents interact with the environment using only the actor network to collect training samples.

All sample data (S, a, r, S') from the agents are centrally stored in the experience replay buffer D for future model training. Through interaction with the environment and training with MADRL, each agent can ultimately construct an optimal strategy for task offloading and resource allocation.

Assume that the number of agents participating in centralized training is denoted as I. The deterministic policies and network parameters of the participants are represented by $\mu = \{\mu_{\theta_1}, \ldots, \mu_{\theta_I}\}$ (abbreviated as μ_i), where $\theta = \{\theta_1, \ldots, \theta_I\}$ denotes the network parameters associated with each participant.

To optimize the policies, the deterministic policy gradient for the i-th agent is formulated as:

$$\nabla_{\theta_i} J(\mu_{\theta_i}) = \mathbb{E}_{S\sim D}\Big[\nabla_{a_e} Q_{\mu_e}(S, a_1, \ldots, a_I) \cdot \nabla_{\theta_i}\mu_{\theta_i}(S_i)\Big], \tag{41}$$

where $S \sim D$ indicates that the states are sampled from the experience replay buffer D, and $\mu_{\theta_i}(S_i)$ represents the deterministic policy generated by the actor network for the state S_i.

The target Q-value, used for updating the critic network, is computed as:

$$y = r_i + \gamma Q_{\mu'_i}(S', a'_1, \ldots, a'_I), \tag{42}$$

where r_i is the reward, γ is the discount factor, $Q_{\mu'_i}$ is the target Q-value function, S' represents the next state, and $a'_1, \ldots, a'_I$ are the target actions.

The loss function for the critic network is defined as:

$$L(\theta_i) = \frac{1}{X}\sum_{j=1}^{X}\left(y_j - Q_{\mu_i}(S_j, a_{1,j}, \ldots, a_{I,j})\right)^2, \tag{43}$$

where X is the batch size, and j indexes the samples in the batch.

The actor network parameters are updated via gradient ascent using the following rule:

$$\theta_i \leftarrow \theta_i + \alpha \nabla_{\theta_i} J(\mu_{\theta_i}), \tag{44}$$

where α is the learning rate. The detailed training process is outlined in Algorithm 1.

The proposed MADRL framework adopts a centralized training and distributed execution strategy. During centralized training, a shared experience replay buffer D is utilized to stabilize learning by decorrelating samples. In the distributed execution phase, agents independently interact with the environment, leveraging their learned policies to make decisions for task offloading and resource allocation. The overall framework of MADRL agents is illustrated in Fig. 2. This combination ensures both training efficiency and scalability in dynamic vehicular networks.

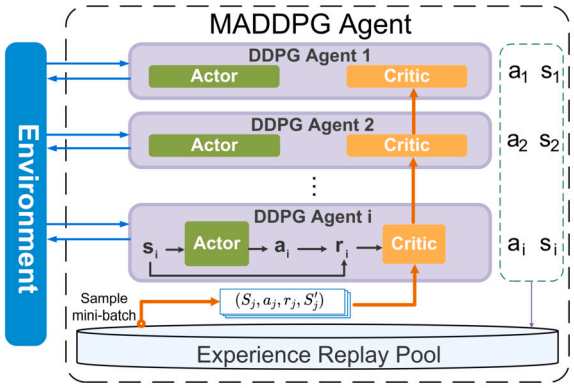


Fig. 2. The overall framework of MADRL agents.

4.5. Computational complexity analysis

Computational complexity serves as a critical metric for assessing the efficiency and hardware compatibility of algorithms, particularly in the context of deep learning and model optimization. Typically, computational complexity is measured in terms of floating-point operations (FLOPs), which represent the total number of floating-point calculations, such as addition, multiplication, and division. FLOPs are widely employed in hardware performance benchmarking and algorithmic optimization, especially in high-performance computing and artificial intelligence applications.

In this section, FLOPs are adopted as the primary metric to analyze the computational demands of the proposed algorithm. For fully connected layers, which involve both weight and bias computations, the FLOPs can be defined as $2 \times M \times N$, where M and N denote the number of input and output neurons, respectively.

The proposed model consists of several sub-network modules with two distinct architectural designs. Specifically, the critic network adopts a four-layer fully connected structure, while the actor network is built with five fully connected layers. The overall computational complexity of the algorithm is determined by the structural parameters of these neural network layers and can be expressed as:

$$Complexity = 2 \times \sum_{n=1}^{5} M_{actor,n} N_{actor,n} + 2 \times \sum_{n=1}^{4} M_{critic,n} N_{critic,n}. \quad (45)$$

In this expression, $M_{actor,n}$ and $N_{actor,n}$ denote the number of input and output neurons in the n-th layer of the actor network, respectively. Similarly, $M_{critic,n}$ and $N_{critic,n}$ represent the corresponding dimensions in the critic network. These parameters collectively define the computational workload required for the matrix multiplications and bias additions in each layer.

Assuming a single-core processor with a computational capacity of 2G FLOPs per second, the proposed model can be executed efficiently on the available hardware. This assessment highlights the algorithm's relatively low computational complexity, coupled with its high operational efficiency, making it well-suited for deployment on resource-constrained devices.

5. Simulation results

5.1. Parameter settings

This paper implements the proposed MADRL-based task offloading and resource allocation algorithm using PyTorch. The simulation environment emulates a typical vehicular network scenario with one cloud server and 4 MEC servers. Each MEC server serves 20 vehicles within its communication range, capturing the complexity of multi-user, multi-server cooperative task offloading.

Table 1
System Parameters Settings.

Parameter	Value
Network Architecture	
Road length	500 m
Number of vehicles	80
Vehicle speed range	10–15 m/s
Vehicle initial positions	Random within road network
Communication Parameters	
Noise power spectral density	-174 dBm/Hz
MEC antenna gain	8 dBi
Vehicle antenna gain	3 dBi
Path loss exponent	3.5
Channel bandwidth	10 MHz
Transmission power	23 dBm
Shadow fading	Log-normal (0, 8 dB)
Computing Parameters	
MEC CPU frequency	6 GHz
Maximum CPU frequency	400 MHz
CPU cycles per bit	1000 cycles/bit
Energy coefficient	10^{-27}
Learning Parameters	
State dimension	14
Action dimension	2
Learning rate	0.001
Discount factor	0.9
Reward scaling factor	0.01

The proposed algorithm employs neural networks for both the actor and critic networks. The actor network consists of three fully connected hidden layers with ReLU activations, and a tanh output layer to constrain actions within the range $[-1, 1]$. The critic network consists of four fully connected hidden layers with ReLU activations, which evaluate the value of the current policy and guide the learning process.

To evaluate the algorithm, it is compared against the following baseline methods:

1. **Local Processing (LP):** All tasks are processed locally, avoiding communication delays but limited by device resources, leading to inefficiencies for computation-intensive tasks.
2. **All Offloading (AO):** Tasks are randomly assigned to edge servers or the cloud, enhancing speed in high-resource scenarios but often causing resource-task mismatches and bottlenecks.
3. **Deep Deterministic Policy Gradient (DDPG):** A reinforcement learning approach with an Actor-Critic structure. While effective, it faces stability and convergence issues in multi-agent scenarios.

The simulation parameters and training configurations in Table 1 demonstrate the proposed algorithm's adaptability and efficiency under diverse task and resource conditions.

5.2. Convergence analysis

Fig. 3 illustrates the accumulated rewards of different algorithms over training episodes, highlighting their performance in task offloading and resource allocation. The proposed MADRL algorithm exhibits significantly faster and more stable convergence compared to DDPG, AO, and LP, reflecting its superior adaptability and efficiency in dynamic vehicular environments.

In the early episodes, the proposed algorithm demonstrates a sharp increase in reward, surpassing the baselines by approximately 50% at episode 200 and stabilizing around 800 by episode 300. This rapid improvement is attributed to its efficient exploration-exploitation mecha-

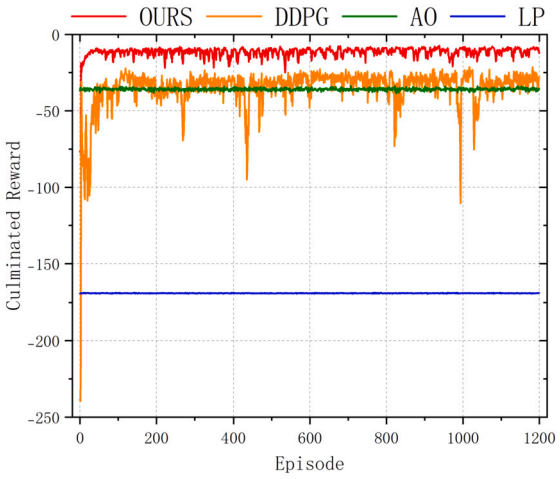
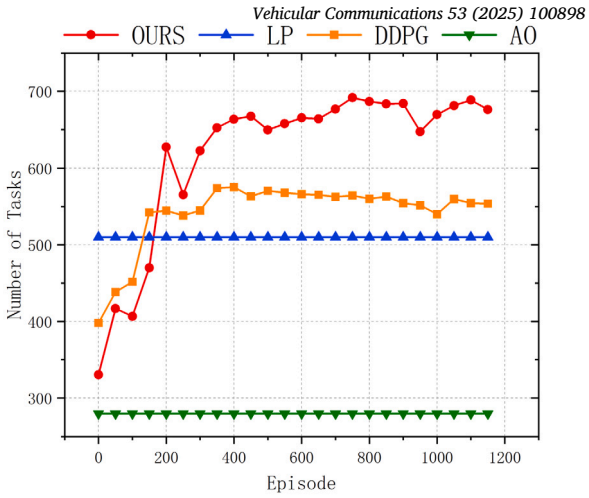
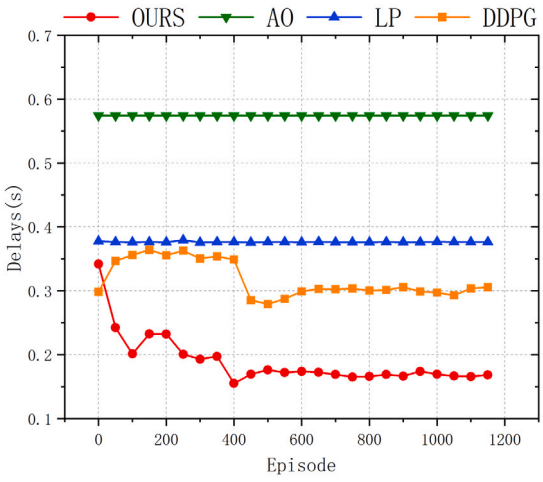


Fig. 3. Comparison of Accumulated Reward Among Different Algorithms.



(a) Task Completion Count



(b) Task Delay

Fig. 4. Comparison of Task Completion Count and Task Delay Among Different Algorithms.

nism, which allows it to quickly identify effective strategies. In contrast, DDPG converges more slowly, with notable fluctuations, due to the challenges of managing inter-agent interactions in a dynamic multi-agent environment.

AO and LP maintain almost constant rewards, with LP achieving the lowest performance due to its reliance on local resources, which are often insufficient for computation-intensive tasks. AO performs slightly better but lacks adaptability, leading to inefficiencies when task requirements and resource availability are mismatched.

This convergence analysis underscores the importance of dynamic learning-based approaches in optimizing task offloading. The proposed algorithm's ability to rapidly converge and achieve higher rewards makes it a robust solution for real-world vehicular networks, where quick adaptability and stability are essential for maintaining system performance.

5.3. Task completion count and task delay analysis

Fig. 4 compares the task completion count and task delay across different algorithms, highlighting the proposed algorithm's superior performance in balancing computational efficiency and timeliness.

5.3.1. Task completion count

As shown in Fig. 4(a), the proposed algorithm demonstrates a rapid increase in task completion count during the early training episodes, stabilizing at approximately 650 tasks as the training progresses. This significant improvement stems from its ability to dynamically allocate tasks based on real-time resource availability and task deadlines, ensuring optimal utilization of resources.

In comparison, DDPG shows a slower rise, reaching around 550 tasks after 400 episodes. The delay in achieving a stable task completion count is attributed to DDPG's difficulty in coordinating policies among agents in a multi-agent setting, leading to inconsistent decisions and suboptimal task assignments during early episodes.

AO and LP perform significantly worse, stabilizing at 280 and 510 tasks, respectively. AO's random offloading often results in mismatches between task requirements and available resources, limiting its efficiency. LP, constrained by local processing capabilities, struggles to handle tasks requiring intensive computations, further reducing its task completion count.

5.3.2. Task delay

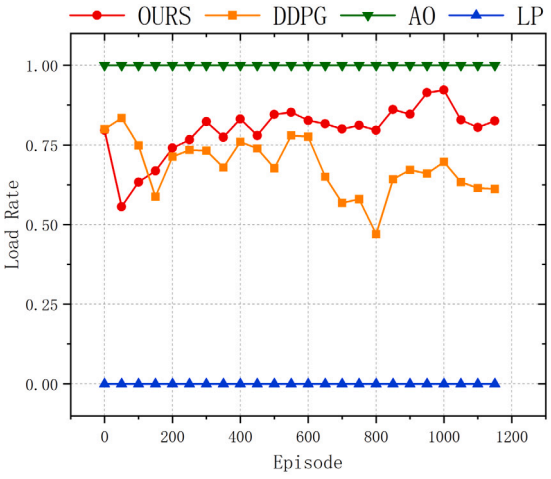
Fig. 4(b) illustrates the task delay for different algorithms. The proposed algorithm achieves the lowest average delay, stabilizing at around 0.2 seconds. This demonstrates its ability to prioritize latency-sensitive tasks and balance resource allocation effectively. The slight fluctuations in the initial episodes are due to the exploration phase, where the algorithm refines its offloading and resource allocation strategies.

In contrast, DDPG exhibits higher delays, fluctuating up to 0.4 seconds. This variability reflects the instability of its learning process in dynamic environments, where frequent state changes and inter-agent dependencies make it challenging to converge on optimal policies.

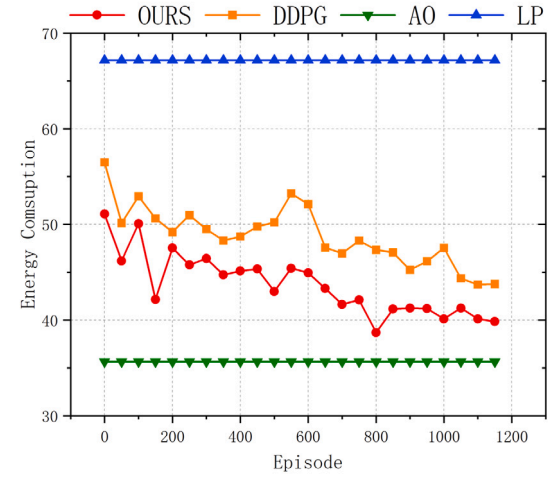
AO and LP, with delays of 0.6 and 0.4 seconds, respectively, are hindered by their static decision-making approaches. AO's lack of adaptability leads to inefficient resource utilization, while LP's reliance on local resources results in processing bottlenecks and prolonged delays.

5.3.3. Insights from analysis

The comparison underscores the importance of dynamic and learning-based strategies in vehicular networks. The proposed algorithm's ability to achieve higher task completion rates while maintaining minimal delay demonstrates its suitability for environments requiring both efficiency and real-time responsiveness. By dynamically adapting to system states and prioritizing tasks based on deadlines and resource availability, it ensures robust performance in diverse scenarios.



(a) Load Rate



(b) Energy Consumption

Fig. 5. Comparison of Load Rate and Energy Consumption Among Different Algorithms.

5.4. Load rate and energy consumption analysis

Fig. 5 illustrates the load rate and energy consumption of different algorithms, providing insights into resource utilization efficiency and energy management.

5.4.1. Load rate

As shown in Fig. 5(a), the proposed algorithm maintains a consistently high load rate, stabilizing close to 0.85. This reflects its ability to fully utilize available resources, ensuring that both computational and communication capacities are effectively employed under dynamic system conditions. By dynamically adjusting task offloading and resource allocation strategies, the proposed algorithm avoids underutilization and maintains system balance even in the presence of varying workloads.

AO achieves a comparable load rate but does so through static resource allocation. While it appears efficient in terms of overall utilization, it lacks the flexibility to adapt to dynamic task demands, potentially leading to inefficiencies when tasks with diverse requirements are processed.

DDPG, on the other hand, exhibits fluctuations in load rate, occasionally dropping below 0.65. This instability can be attributed to inconsistent policy updates and the challenges of coordinating multiple agents in a dynamic environment. LP consistently shows the lowest load rate, close to 0.0, as it relies entirely on local resources, which are insufficient to handle high-demand tasks effectively.

5.4.2. Energy consumption

Fig. 5(b) highlights the energy consumption trends of the different algorithms. The proposed algorithm achieves the most efficient energy usage, stabilizing around 40 units after episode 200. This represents a 10% reduction compared to DDPG, demonstrating its ability to minimize high-energy operations such as unnecessary offloading to distant servers or overloading individual resources.

LP, on the other hand, consumes the highest energy due to its reliance on local computation, which is less energy-efficient for processing intensive tasks.

DDPG's energy consumption remains higher than the proposed algorithm, fluctuating due to its less stable task allocation policies. These fluctuations indicate that while DDPG can occasionally make efficient decisions, its overall energy management is inconsistent, leading to periods of higher consumption.

While AO achieves the lowest energy consumption among all methods by avoiding frequent task processing, this comes at a significant cost. Specifically, the reduced task completion rates and increased delays associated with AO's approach can severely impact the overall system performance. In practical applications, such as real-time vehicular networks, the ability to complete tasks promptly is often more critical than minimizing energy consumption. For instance, delays in processing critical tasks like navigation updates or collision avoidance alerts can lead to safety hazards or inefficient routing, which may negate the benefits of lower energy usage. Moreover, the reduced task completion rates can result in a backlog of unprocessed tasks, further exacerbating delays and potentially leading to system instability. Consequently, although AO's energy efficiency is noteworthy, its practical relevance is diminished due to the detrimental effects on task completion and system responsiveness.

5.4.3. Insights from analysis

The consistently high load rate and low energy consumption achieved by the proposed algorithm demonstrate its ability to balance resource utilization and energy efficiency effectively. These attributes are critical in vehicular networks, where energy is a limited resource, and maintaining high system throughput is essential. By dynamically adapting to changing system states, the proposed algorithm ensures efficient operation without compromising task completion rates or latency, making it a robust solution for real-world applications.

5.5. Comprehensive system cost analysis

To further demonstrate the scalability of the proposed algorithm in ultra-dense vehicular networks and provide a more comprehensive evaluation of its performance, this section introduces a comprehensive system cost metric. This metric combines the four basic metrics of task completion count, task delay, energy consumption, and load rate, allowing for a more nuanced assessment of the algorithms' performance in complex and dynamic environments.

The comprehensive system cost is defined as follows:

$$\text{Cost} = \omega_1 \cdot \text{Count} + \omega_2 \cdot \text{Delay} + \omega_3 \cdot \text{Energy} + \omega_4 \cdot \text{Load} \tag{46}$$

where $\omega_1, \omega_2, \omega_3, \omega_4$ are the weights assigned to each metric, satisfying $\sum_{i=1}^{4} \omega_i = 1$.

To ensure fair comparison and highlight the differences among algorithms, the comprehensive system cost values were normalized to a range of [0, 1].

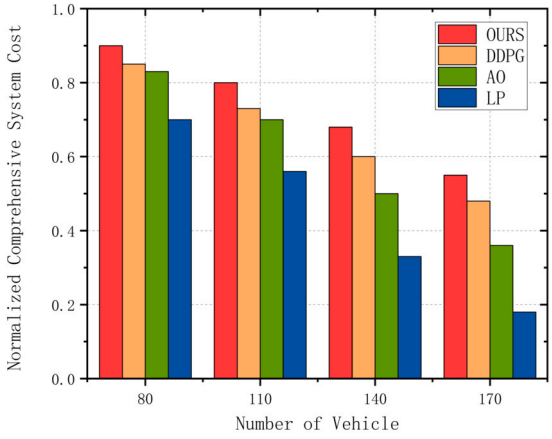


Fig. 6. Normalized comprehensive system cost of different algorithms.

As shown in Fig. 6, the proposed MADRL algorithm consistently outperforms the other three algorithms across all vehicle counts. For instance, when the number of vehicles is 80, the normalized comprehensive system cost of MADRL is 0.9, while the costs for LP, AO, and DDPG are 0.7, 0.83, and 0.85, respectively. This trend continues as the number of vehicles increases. When the number of vehicles reaches 170, the normalized comprehensive system cost of MADRL is 0.55, while the costs for LP, AO, and DDPG are 0.18, 0.36, and 0.48, respectively.

The superior performance of the MADRL algorithm can be attributed to its ability to dynamically adjust task offloading and resource allocation strategies based on real-time network conditions and task requirements. This adaptability ensures that the algorithm can efficiently handle the increasing complexity and resource demands of ultra-dense vehicular networks.

In contrast, the LP algorithm, which relies solely on local resources, shows a significant increase in the normalized comprehensive system cost as the number of vehicles increases. This is due to the limited computational and communication resources available on individual vehicles, which become overwhelmed as the network density increases.

The AO algorithm, which randomly offloads tasks to edge servers or the cloud, also shows an increase in the normalized comprehensive system cost with the number of vehicles. This is because the random offloading strategy often leads to suboptimal resource utilization and increased delays, especially in dense networks where the demand for resources is high.

The DDPG algorithm, while showing better performance than LP and AO, still falls short of the MADRL algorithm. The DDPG algorithm struggles to adapt to the dynamic and complex environment of vehicular networks, leading to suboptimal task offloading and resource allocation decisions.

Overall, the proposed MADRL algorithm shows a significant advantage over the other algorithms, especially as the number of vehicles increases. This indicates that the proposed algorithm is not only effective in small-scale scenarios but also scalable to large-scale vehicular networks, making it a robust solution for real-world applications.

5.6. Summary of results

The results demonstrate that the proposed algorithm consistently outperforms baseline methods across all evaluation metrics. Its adaptive offloading and resource allocation capabilities enable it to achieve higher rewards, increased task completion counts, lower delays, higher resource utilization, and reduced energy consumption. These findings highlight the proposed algorithm's effectiveness in addressing the challenges of task offloading and resource management in vehicular networks.

6. Conclusion

This paper addresses the challenges of task offloading and resource allocation in vehicular networks through a cloud-edge-device collaborative architecture. Leveraging MADRL, the proposed algorithm effectively balances computational efficiency, energy consumption, and task latency in dynamic vehicular environments.

The study shows that the proposed algorithm consistently outperforms baseline methods across key performance metrics, including task completion rate, delay, load rate, and energy consumption. The dynamic offloading and adaptive resource allocation capabilities of the proposed framework lead to significant improvements in task processing efficiency and overall system stability, even in resource-constrained and dynamic vehicular networks.

However, the scalability of the algorithm in ultra-dense vehicular networks and its performance under varying traffic patterns and mobility models require further investigation. Additionally, while the current framework focuses on efficiency and resource optimization, future work could extend the algorithm to address security-related challenges, such as secure task offloading, data privacy protection, and resilience against cyber-attacks in vehicular networks. As highlighted in [2], "integrating security capabilities with sensing and communication is crucial for future 6G systems," and the proposed framework could be extended to incorporate artificial noise generation and beamforming techniques to enhance physical layer security. The integration of advanced communication technologies, such as 6G and V2X, could further enhance real-time decision-making capabilities and support secure and reliable operations in next-generation intelligent transportation systems.

In conclusion, this study provides a foundation for intelligent task offloading and resource allocation in vehicular networks. The MADRL-based framework offers a promising solution for managing dynamic resources in next-generation intelligent transportation systems and sets the stage for future advancements in the field.

CRediT authorship contribution statement

Peiying Zhang: Conceptualization. **Enqi Wang:** Writing – original draft. **Lizhuang Tan:** Conceptualization. **Neeraj Kumar:** Conceptualization. **Jian Wang:** Conceptualization. **Kai Liu:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

References

[1] B.S. Babu, N. Jayashree, Edge intelligence models for industrial IoT (IIoT), RV J. Sci. Technol. Eng. Art. Manag. 1 (2020) 5–17.

[2] A. Bazzi, M. Chafii, Secure full duplex integrated sensing and communications, IEEE Trans. Inf. Forensics Secur. 19 (2024) 2082–2097, https://doi.org/10.1109/TIFS.2023.3346696.

[3] H. Guo, J. Liu, J. Ren, Y. Zhang, Intelligent task offloading in vehicular edge computing networks, IEEE Wirel. Commun. 27 (2020) 126–132, https://doi.org/10.1109/MWC.001.1900489.

[4] Y. Li, X. Wang, X. Gan, H. Jin, L. Fu, X. Wang, Learning-aided computation offloading for trusted collaborative mobile edge computing, IEEE Trans. Mob. Comput. 19 (2020) 2833–2849, https://doi.org/10.1109/TMC.2019.2934103.

[5] B. Lin, K. Lin, C. Lin, Y. Lu, Z. Huang, X. Chen, Computation offloading strategy based on deep reinforcement learning for connected and autonomous vehicle in vehicular edge computing, IEEE Cloud Comput. (2021), https://doi.org/10.1186/s13677-021-00246-6.

[6] J. Liu, J. Wan, B. Zeng, Q. Wang, H. Song, M. Qiu, A scalable and quick-response software defined vehicular network assisted by mobile edge computing, IEEE Commun. Mag. 55 (2017) 94–100, https://doi.org/10.1109/MCOM.2017.1601150.

[7] Y. Liu, S. Wang, J. Huang, F. Yang, A computation offloading algorithm based on game theory for vehicular edge networks, in: 2018 IEEE International Conference on Communications (ICC), 2018, pp. 1–6.
[8] Y. Liu, H. Yu, S. Xie, Y. Zhang, Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks, IEEE Trans. Veh. Technol. 68 (2019) 11158–11168, https://doi.org/10.1109/TVT.2019.2935450.
[9] X.Q. Pham, T. Huynh-The, E.N. Huh, D.S. Kim, Partial computation offloading in parked vehicle-assisted multi-access edge computing: a game-theoretic approach, IEEE Trans. Veh. Technol. 71 (2022) 10220–10225, https://doi.org/10.1109/TVT.2022.3182378.
[10] G. Plastiras, M. Terzi, C. Kyrkou, T. Theocharides, Edge intelligence: challenges and opportunities of near-sensor machine learning applications, in: 2018 IEEE 29th International Conference on Application-Specific Systems, Architectures and Processors (ASAP), 2018, pp. 1–7.
[11] G. Qiao, S. Leng, K. Zhang, Y. He, Collaborative task offloading in vehicular edge multi-access networks, IEEE Commun. Mag. 56 (2018) 48–54, https://doi.org/10.1109/MCOM.2018.1701130.
[12] V. Sethi, S. Pal, Feddove: a federated deep q-learning-based offloading for vehicular fog computing, Future Gener. Comput. Syst. 141 (2023) 96–105, https://doi.org/10.1016/j.future.2022.11.012.
[13] F. Sun, F. Hou, N. Cheng, M. Wang, H. Zhou, L. Gui, X. Shen, Cooperative task scheduling for computation offloading in vehicular cloud, IEEE Trans. Veh. Technol. 67 (2018) 11049–11061, https://doi.org/10.1109/TVT.2018.2868013.
[14] Y. Sun, X. Guo, S. Zhou, Z. Jiang, X. Liu, Z. Niu, Learning-based task offloading for vehicular cloud computing systems, in: 2018 IEEE International Conference on Communications (ICC), 2018, pp. 1–7.
[15] L.T. Tan, R.Q. Hu, L. Hanzo, Twin-timescale artificial intelligence aided mobility-aware edge caching and computing in vehicular networks, IEEE Trans. Veh. Technol. 68 (2019) 3086–3099, https://doi.org/10.1109/TVT.2019.2893898.
[16] K. Wang, X. Wang, X. Liu, A high reliable computing offloading strategy using deep reinforcement learning for IOVS in edge computing, J. Grid Comput. 19. URL (2021), https://doi.org/10.1007/s10723-021-09542-6.
[17] Y. Wang, P. Lang, D. Tian, J. Zhou, X. Duan, Y. Cao, D. Zhao, A game-based computation offloading method in vehicular multiaccess edge computing networks, IEEE Internet Things J. 7 (2020) 4987–4996, https://doi.org/10.1109/JIOT.2020.2972061.
[18] Z. Wang, D. Zhao, M. Ni, L. Li, C. Li, Collaborative mobile computation offloading to vehicle-based cloudlets, IEEE Trans. Veh. Technol. 70 (2021) 768–781, https://doi.org/10.1109/TVT.2020.3043296.
[19] Z. Wang, Z. Zhong, M. Ni, A semi-Markov decision process-based computation offloading strategy in vehicular networks, in: 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC), 2017, pp. 1–6.
[20] Y. Wu, L.P. Qian, H. Mao, X. Yang, H. Zhou, X. Tan, D.H.K. Tsang, Secrecy-driven resource management for vehicular computation offloading networks, IEEE Netw. 32 (2018) 84–91, https://doi.org/10.1109/MNET.2018.1700320.
[21] J. Xu, L. Chen, P. Zhou, Joint service caching and task offloading for mobile edge computing in dense networks, in: IEEE INFOCOM 2018 - IEEE Conference on Computer Communications, 2018, pp. 207–215.
[22] X. Xu, K. Liu, P. Dai, F. Jin, H. Ren, C. Zhan, S. Guo, Joint task offloading and resource optimization in noma-based vehicular edge computing: a game-theoretic drl approach, J. Syst. Archit. 134 (2023) 102780, https://doi.org/10.1016/j.sysarc.2022.102780.
[23] X. Xu, Y. Xue, X. Li, L. Qi, S. Wan, A computation offloading method for edge computing with vehicle-to-everything, IEEE Access 7 (2019) 131068–131077, https://doi.org/10.1109/ACCESS.2019.2940295.
[24] X. Xu, Y. Xue, L. Qi, Y. Yuan, X. Zhang, T. Umer, S. Wan, An edge computing-enabled computation offloading method with privacy preservation for Internet of connected vehicles, Future Gener. Comput. Syst. 96 (2019) 89–100, https://doi.org/10.1016/j.future.2019.01.012.
[25] C. Yang, X. Xu, X. Zhou, L. Qi, Deep q network–driven task offloading for efficient multimedia data analysis in edge computing–assisted IOV, ACM Trans. Multimed. Comput. Commun. Appl. 18 (2022), https://doi.org/10.1145/3548687.
[26] L. Yang, H. Zhang, M. Li, J. Guo, H. Ji, Mobile edge computing empowered energy efficient task offloading in 5G, IEEE Trans. Veh. Technol. 67 (2018) 6398–6409, https://doi.org/10.1109/TVT.2018.2799620.
[27] D. Zhang, L. Cao, H. Zhu, T. Zhang, J. Du, K. Jiang, Task offloading method of edge computing in Internet of vehicles based on deep reinforcement learning, Clust. Comput. 25 (2022) 1175–1187, https://doi.org/10.1007/s10586-021-03532-9.
[28] K. Zhang, Y. Mao, S. Leng, S. Maharjan, Y. Zhang, Optimal delay constrained offloading for vehicular edge computing networks, in: 2017 IEEE International Conference on Communications (ICC), 2017, pp. 1–6.
[29] Y. Zhang, M. Zhang, C. Fan, F. Li, B. Li, Computing resource allocation scheme of IOV using deep reinforcement learning in edge computing environment, EURASIP J. Adv. Signal Process. 2021 (2021), https://api.semanticscholar.org/CorpusID:233696262.
[30] K. Zheng, H. Meng, P. Chatzimisios, L. Lei, X. Shen, An smdp-based resource allocation in vehicular cloud computing systems, IEEE Trans. Ind. Electron. 62 (2015) 7920–7928, https://doi.org/10.1109/TIE.2015.2482119.

Peiying Zhang is currently an Associate Professor with the College of Computer Science and Technology, China University of Petroleum (East China). He received his Ph.D. in the School of Information and Communication Engineering at University of Beijing University of Posts and Telecommunications in 2019. He has published multiple IEEE/ACM Trans./Journal/Magazine papers since 2016, such as IEEE TII, IEEE T-ITS, IEEE TVT, IEEE TNSE, IEEE TNSM, IEEE TETC, IEEE Network and etc. He served as the Technical Program Committee of AAAI'24, AAAI'23, IEEE ICC'23, IEEE ICC'22, and INFOCOM Wireless-Sec 2023. He is the Leading Guest Editor of Drones, Mathematics, Electronics, Wireless Communications and Mobile Computing, and etc. He is the editorial board of Drones, CMC-Computers, Materials & Continua, Mobile Information Systems, International Journal of Computational Intelligence Systems and Artificial Intelligence and Applications (AIA). His research interests include semantic computing, future internet architecture, network virtualization, and artificial intelligence for networking.

Enqi Wang is currently pursuing a master's degree in the School of Computer Science and Technology, China University of Petroleum (East China). He received a bachelor's degree in Network Engineering from Linyi University. His research interests include resource allocation and artificial intelligence for networking.

Lizhuang Tan is currently an Associate Professor with Shandong Provincial Key Laboratory of Computer Networks, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences). He received his Ph.D. degree from School of Electronic and Information Engineering, Beijing Jiaotong University in 2022. He has published more than 20 journal or conference papers, such as IEEE TCC/TNSM/TVT/IOTJ, ELSEVIER COMNET/COMCOM, APNOMS, etc. His research interests include network measurement, management and optimization, especially software-defined networking and data center networking.

Neeraj Kumar (Senior Member, IEEE) is currently working as a Full Professor with the Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala, India. He has published more than 400 technical research papers, which are cited more than 43637 times by researchers across the globe with a current H-index of 114. His broad research areas are green computing and network management, IoT, big data analytics, deep learning, and cyber-security. He has also edited/authored ten books with international/national publishers. He is a highly-cited researcher from WoS from 2019 to 2021. Prof. Kumar has organized various special issues of journals of repute from IEEE, Elsevier, and Springer. He has been the Workshop Chair at IEEE GLOBECOM 2018, IEEE INFOCOM 2020, and IEEE ICC 2020, and the track chair. He is serving as an Editor of ACM Computing Surveys, IEEE Transactions on Services Computing, IEEE Transactions on Network and Service Management, IEEE Network Magazine, IEEE Communications Magazine, Journal of Networks and Computer Applications (Elsevier), Computer Communications (Elsevier), and International Journal of Communication Systems (Wiley).

Jian Wang is currently a Professor and serves as the Head of the Laboratory for Intelligent Information Processing with the College of Science, China University of Petroleum (East China). His research interests include computational intelligence, differential programming, clustering, fuzzy systems, evolutionary computation. Prof. Wang serves as an Associate Editor for the IEEE Transactions on Neural Networks and Learning Systems, IEEE Transactions on Emerging Topics in Computational Intelligence, Information Sciences, International Journal of Machine Learning and Cybernetics, and Journal of Applied Computer Science Methods. He also serves on the Editorial Board for the Neural Computing & Applications and Complex & Intelligent Systems.

Kai Liu received the B.S. and M.S. degrees from Xidian University, Xi'an, China, in 2009 and 2012, respectively, and the Ph.D. degree from Tsinghua University, Beijing, China, in 2016. He is currently an Assistant Research Fellow with the Beijing National Research Center for Information Science and Technology, Tsinghua University. His current research interests include space information networks and on-board switching.